Status	Finished
Started	Wednesday, 21 October 2026, 9:56 PM
Completed	Wednesday, 21 October 2026, 10:05 PM
Duration	9 mins 47 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
def find_longest_word(filename):
    longest_word = ""

    with open(filename, "r") as file:
        for line in file:
            # Remove punctuation attached to words
            words = "".join(
                char if char.isalnum() or char.isspace() else " "
                for char in line
            ).split()
            for word in words:
                if len(word) > len(longest_word):
                    longest_word = word

    print(f"Longest word: {longest_word}")


if __name__ == "__main__":
    filename = input().strip()
    find_longest_word(filename)
```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import os
import sys
from collections import Counter


def count_word_frequencies():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Read from file if input_data is an existing file path, otherwise treat as raw text
    if os.path.isfile(input_data):
        with open(input_data, "r") as file:
            text = file.read()
    else:
        text = input_data

    # Extract words clean of punctuation while converting to lowercase
    words = "".join(
        char.lower() if char.isalnum() or char.isspace() else " " for char in text
    ).split()

    if not words:
        return

    # Count frequencies of each word
    counts = Counter(words)

    # Sort alphabetically by word and write results to output.txt
    with open("output.txt", "w") as file:
        for word in sorted(counts.keys()):
            file.write(f"{word}: {counts[word]}\n")


if __name__ == "__main__":
    count_word_frequencies()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import os
import sys


def find_palindromes():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Check if input is a filename that exists, otherwise treat input as text content
    if os.path.isfile(input_data):
        with open(input_data, "r") as file:
            text = file.read()
    else:
        text = input_data

    # Extract words clean of non-alphanumeric characters
    words = "".join(
        char if char.isalnum() or char.isspace() else " " for char in text
    ).split()

    palindromes = []
    seen = set()

    for word in words:
        clean_word = word.lower()
        # A palindrome reads the same forwards and backwards
        if clean_word == clean_word[::-1] and word not in seen:
            palindromes.append(word)
            seen.add(word)

    # Write found palindromes to output.txt
    with open("output.txt", "w") as file:
        for item in palindromes:
            file.write(f"{item}\n")


if __name__ == "__main__":
    find_palindromes()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import os
import sys


def delete_line():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    first_arg = input_data[0].strip()

    # Determine if input starts with a valid existing filename or raw text lines
    if os.path.isfile(first_arg):
        file_path = first_arg
        line_to_delete = int(input_data[1].strip())
        with open(file_path, "r") as file:
            lines = file.readlines()
    else:
        # Separate content lines from the trailing line number integer
        line_to_delete = int(input_data[-1].strip())
        lines = [line + "\n" for line in input_data[:-1]]

    # Delete the specified 1-based line index
    updated_lines = [
        line for i, line in enumerate(lines, start=1) if i != line_to_delete
    ]

    # Write the result to output.txt
    with open("output.txt", "w") as file:
        file.writelines(updated_lines)


if __name__ == "__main__":
```

```
34      delete_line()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
import os
import sys


def read_specific_line():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    first_arg = input_data[0].strip()

    # Determine if input starts with a valid existing filename or raw text lines
    if os.path.isfile(first_arg):
        file_path = first_arg
        line_num = int(input_data[1].strip())
        with open(file_path, "r") as file:
            lines = file.readlines()
    else:
        line_num = int(input_data[-1].strip())
        lines = input_data[:-1]

    # Print the line if 1-based index is within bounds
    if 1 <= line_num <= len(lines):
        print(lines[line_num - 1].rstrip("\r\n"))


if __name__ == "__main__":
    read_specific_line()
```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
import os
import sys


def count_words_in_file():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Extract the first line as the filename/input
    lines = input_data.splitlines()
    first_arg = lines[0].strip()

    # Read from file if valid existing path, otherwise process standard input text
    if os.path.isfile(first_arg):
        with open(first_arg, "r") as file:
            text = file.read()
    else:
        text = input_data

    # Clean punctuation and split into words
    words = "".join(
        char if char.isalnum() or char.isspace() else " " for char in text
    ).split()

    print(f"Total words: {len(words)}")


if __name__ == "__main__":
    count_words_in_file()
```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
	`Inserted line.`
		`Line one.`
		`Line two.`
		`Inserted line.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import os
import sys


def insert_line_at_position():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    first_arg = input_data[0].strip()

    if os.path.isfile(first_arg):
        file_path = first_arg
        line_num = int(input_data[1].strip())
        new_content = input_data[2]
        with open(file_path, "r") as file:
            lines = file.readlines()
    else:
        # Standard input format: text lines followed by index and new content
        line_num = int(input_data[-2].strip())
        new_content = input_data[-1]
        lines = [line + "\n" for line in input_data[:-2]]

    # Ensure every existing line ends with a newline
    lines = [
        line if line.endswith("\n") else line + "\n" for line in lines
    ]
```

```
28
29      # Format new line content ensuring trailing newline
30      new_line = (
31          new_content if new_content.endswith("\n") else new_content + "\n"
32      )
33
34      # Insert at 1-based line position (index = line_num - 1)
35      insert_index = max(0, min(line_num - 1, len(lines)))
36      lines.insert(insert_index, new_line)
37
38      # Write output to output.txt
39      with open("output.txt", "w") as file:
40          file.writelines(lines)
41
42
43  if __name__ == "__main__":
44      insert_line_at_position()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
		Inserted line.
			Line one.
			Line two.
			Inserted line.
			Line three.
			Line four.
				Line one.
				Line two.
				Inserted line.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		4
		Inserted line D.
			Line A.
			Line B.
			Line C.
			Inserted line D.
				Line A.
				Line B.
				Line C.
				Inserted line D.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import os
import sys


def replace_line():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    first_arg = input_data[0].strip()

    if os.path.isfile(first_arg):
        file_path = first_arg
        line_num = int(input_data[1].strip())
        new_content = input_data[2]
        with open(file_path, "r") as file:
            lines = file.readlines()
    else:
        line_num = int(input_data[-2].strip())
        new_content = input_data[-1]
        lines = [line + "\n" for line in input_data[:-2]]

    # Ensure each existing line ends with a newline character
    lines = [
        line if line.endswith("\n") else line + "\n" for line in lines
    ]

    # Format new content with a trailing newline
    new_line = (
```

```
        new_content if new_content.endswith("\n") else new_content + "\n"
    )

    # Replace line at 1-based index if within valid range
    target_index = line_num - 1
    if 0 <= target_index < len(lines):
        lines[target_index] = new_line

    # Write result to output.txt
    with open("output.txt", "w") as file:
        file.writelines(lines)


if __name__ == "__main__":
    replace_line()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
		Updated line two.
			Line one.
			Updated line two.
			Line three.
			Line four.
				Line one.
				Updated line two.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
		Line B Updated.
			Line A.
			Line B Updated.
			Line C.
				Line A.
				Line B Updated.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
		`Line one.`
		`Line two.`
		`eerht eniL.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import os
import sys


def reverse_line():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    first_arg = input_data[0].strip()

    if os.path.isfile(first_arg):
        file_path = first_arg
        line_num = int(input_data[1].strip())
        with open(file_path, "r") as file:
            lines = file.readlines()
    else:
        line_num = int(input_data[-1].strip())
        lines = [line + "\n" for line in input_data[:-1]]

    # Ensure each line ends with a newline
    lines = [
        line if line.endswith("\n") else line + "\n" for line in lines
    ]

    target_index = line_num - 1
    if 0 <= target_index < len(lines):
        line = lines[target_index].rstrip("\r\n")

        # Separate trailing punctuation (like '.') from main text
        punct = ""
        while line and not line[-1].isalnum():
```

```
            punct = line[-1] + punct
            line = line[:-1]

        # Reverse words in sentence order and reverse letters in each word
        words = line.split(" ")
        reversed_words = [word[::-1] for word in reversed(words)]
        reversed_line = " ".join(reversed_words) + punct + "\n"

        lines[target_index] = reversed_line

    # Write output to output.txt
    with open("output.txt", "w") as file:
        file.writelines(lines)


if __name__ == "__main__":
    reverse_line()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`output1.txt`
		`This is the source file.`
		`It contains multiple lines of text.`
		`Here is another line.`

Answer: (penalty regime: 0 %)

```
import os
import sys


def copy_file_content():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # Extract source and destination filenames
    source_file = input_data[0].strip()
    dest_file = input_data[1].strip() if len(input_data) > 1 else "output.txt"

    # Read from source file if it exists, otherwise process standard input text
    if os.path.isfile(source_file):
        with open(source_file, "r") as src:
            content = src.read()
    else:
        content = "\n".join(input_data[2:])
        if content and not content.endswith("\n"):
            content += "\n"

    # Write contents to destination file
    with open(dest_file, "w") as dest:
        dest.write(content)


if __name__ == "__main__":
    copy_file_content()
```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`output1.txt`
			`This is the source file.`
			`It contains multiple lines of text.`
			`Here is another line.`
				`This is the source file.`
				`It contains multiple lines of text.`
				`Here is another line.`
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`output2.txt`
			`Hello, world!`
			`Python programming is amazing.`
			`Let's copy this text to another file.`
				`Hello, world!`
				`Python programming is amazing.`
				`Let's copy this text to another file.`
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input3.txt`
		`output3.txt`
			`Single line.`	`Single line.`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.